Instituto Politécnico do Cávado e do Ave
Escola Superior de Tecnologia

Unidade Curricular: **Integração de Sistemas de Informação (ISI)**

Relatório do Trabalho Prático I

Há campo?

Integração da disponibilidade de campos de ténis de vários clubes com Pentaho Data Integration (Kettle) e PostgreSQL

Identificação

Aluno:	Alexandre Magalhães Martins Machado (N.º 31499)
Curso:	Licenciatura em Engenharia de Sistemas Informáticos (pós-laboral)
Docente:	Óscar Ribeiro
Modalidade:	Trabalho individual

Ano Letivo: **2026/2027**

Barcelos, 22 de outubro de 2026

Conteúdo

1 Enquadramento **2**

2 Problema **2**

3 Estratégia utilizada **3**
3.1 Origens (Extract) . . . 3
3.2 Operadores utilizados (Transform) . . . 3
3.3 Destinos (Load) . . . 3
3.4 Modelo de dados . . . 4

4 Transformações **5**
4.1 T0 – Previsão do tempo (API REST) . . . 5
4.2 T1 – Clube A: caderno do bar (CSV) . . . 5
4.3 T2 – Clube B: aplicação de sócios (JSON) . . . 6
4.4 T3 – Clube C: agenda municipal (XML) . . . 7
4.5 T4 – Consolidar e publicar . . . 8

5 Jobs **10**

6 Vídeo com demonstração **11**

7 Conclusão e trabalhos futuros **11**

8 Referências **12**

Anexo A – Declaração de utilização de inteligência artificial **12**

1 Enquadramento

Tabela 1: Identificação do trabalho.

Equipa	Alexandre Magalhães Martins Machado (31499), trabalho individual
Unidade curricular	Integração de Sistemas de Informação, 3.º ano, 1.º semestre, 2026/2027
Curso	Licenciatura em Engenharia de Sistemas Informáticos, regime pós-laboral, IPCA
Plataformas	Pentaho Data Integration (Kettle) 10.2, PostgreSQL 17, API REST Open-Meteo; página HTML/JavaScript para apresentar o resultado

O enunciado do Trabalho Prático I [1] pede o desenvolvimento de um processo ETL (*Extract, Transform, Load*) completo, com dados de várias origens e formatos, transformações de limpeza e de lógica de negócio e vários destinos. O tema parte de um projeto pessoal, o **hacampo** (*"Há campo?"*): um quadro público que mostra a qualquer jogador que campos de ténis estão livres ou ocupados num clube, sem ser necessário telefonar.

2 Problema

Em muitos clubes de ténis não é possível consultar online se há campos livres. As marcações estão repartidas por sistemas que não comunicam entre si: num clube são escritas num caderno no bar e passadas para Excel; noutro existe uma aplicação apenas para sócios; num complexo municipal estão na agenda da câmara. Quem quer jogar tem de contactar cada um.

O objetivo do trabalho é **integrar as marcações de três clubes num único quadro de disponibilidade**. Para isso é necessário resolver problemas típicos de integração ao nível dos dados, resumidos na Tabela 2.

Tabela 2: Problemas de integração encontrados nos dados.

Problema	**Exemplo nos dados**
Formatos diferentes	CSV com `;`, JSON com objetos aninhados, XML com atributos e validação por XSD
O mesmo valor escrito de várias formas	datas `21/10/2026`, `2026-10-21`, `21-10-26`; horas `18h`, `18:00`, `18.30`, `18H`; campos `C1`, `Campo 1`, `court 1`; telefones `+351 912...`, `00351912...`
Dados incompletos ou inválidos	reserva sem hora, data sem ano, campo que não existe, reserva depois do fecho do clube
Registos repetidos	linha escrita duas vezes no caderno; a aplicação envia a mesma reserva duas vezes (mesmo `id`); reservas canceladas
Dados pessoais	nomes, telefones e emails não podem aparecer num quadro público (RGPD [9])
Informação de outros sistemas	se vai chover num campo descoberto e se já é de noite num campo sem iluminação
Conflitos de negócio	duas reservas no mesmo campo à mesma hora

Os clubes e as reservas são fictícios, gerados com erros intencionais para existir o que limpar. A previsão do tempo é real e obtida da API Open-Meteo [5].

3 Estratégia utilizada

O processo foi implementado no Pentaho Data Integration [3, 4], seguindo os materiais da unidade curricular [2], e foi dividido em cinco transformações orquestradas por um *job* (Figura 1). As três transformações de entrada (T1, T2 e T3) tratam, cada uma, um formato, e escrevem todas para a mesma tabela intermédia (*staging*) já no mesmo formato. A transformação T4 trabalha apenas sobre essa tabela e não precisa de saber de onde veio cada reserva; para juntar um quarto clube basta acrescentar uma nova transformação de entrada.

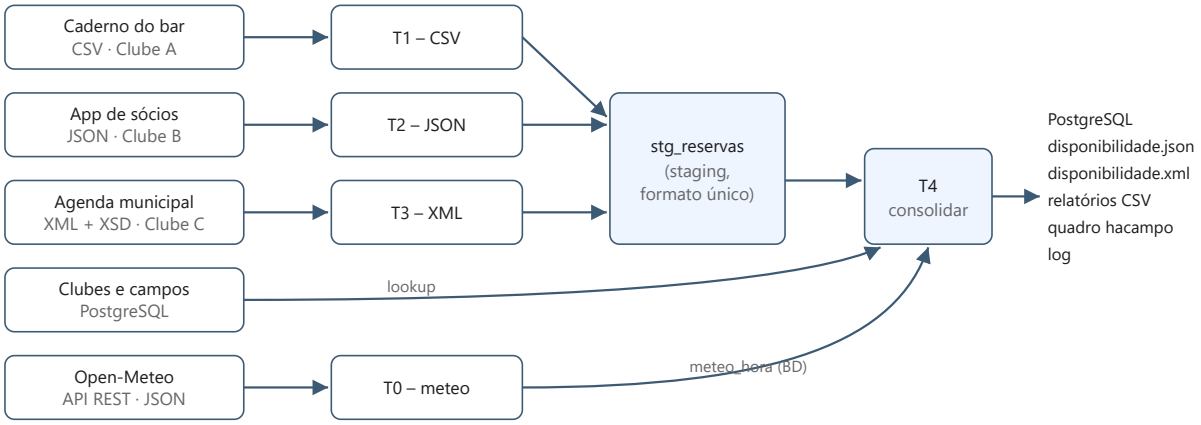


Figura 1: Arquitetura do processo: origens, transformações e destinos.

3.1 Origens (Extract)

Tabela 3: Origens de dados utilizadas.

Origem	Natureza	Formato	Step do Kettle
`clube_a_caderno.csv`	ficheiro local	CSV (`;`)	*CSV file input*
`clube_b_app.json`	ficheiro local	JSON aninhado	*JSON input* (JSONPath)
`clube_c_agenda.xml` + `.xsd`	ficheiro local	XML	*XSD validator* (job) + *Get data from XML* (XPath)
tabelas `clubes` e `campos`	base de dados relacional	PostgreSQL	*Table input*
`api.open-meteo.com`	serviço web remoto	JSON	*HTTP client* + *JSON input*

3.2 Operadores utilizados (Transform)

A Tabela 4 relaciona os critérios de mais-valia do enunciado com os operadores utilizados.

3.3 Destinos (Load)

Os resultados são carregados no PostgreSQL [6] (esquema `hacampo`, tabelas `stg_reservas`, `meteo_hora`, `ocupacao_slot` e `ocupacao_diaria`), no quadro público `disponibilidade.json` e `disponibilidade.xml`, em sete relatórios CSV e no *log* de execução. A página `src/quadro/index.html` lê o JSON e desenha o quadro livre/ocupado (Figura 8).

Tabela 4: Critérios do enunciado e operadores correspondentes.

Critério	**Onde está no trabalho**
Expressões regulares	*Replace in string* (normalizar datas, horas, campos e telefones) e *Regex evaluation* (validar e extrair grupos) em T1, T2 e T3
Nulos e validação	*Filter rows* com `IS NOT NULL` e com os resultados das expressões regulares; linhas inválidas enviadas para ficheiros de erro
Duplicados	*Sort rows* + *Unique rows* com redirecionamento dos repetidos (T2 e T4)
Anonimização	*Add a checksum* SHA-256 do telefone; *Select values* remove nome, email e telefone
Conversão de formatos	CSV, JSON e XML → tabela; tabela → JSON e XML; datas em texto ↔ tipo data
Junções e *lookups*	*Stream lookup* aos campos e horários e à previsão do tempo
Agregações e cálculos	*Calculator* (minutos, duração, n.º de blocos, taxa) e *Group by* (reservas por bloco, ocupação por dia)
Reestruturação	*Split fields*, *Value mapper*, *Clone row* (uma reserva → n blocos), *String operations*, *Add constants*, *Modified JavaScript value*
Orquestração, *logs* e serviços remotos	*job* com tratamento de erros, verificação de ficheiros, validação XSD, SQL, parâmetros, *Write to log* e registo em ficheiro; chamada à API REST

3.4 Modelo de dados

A Listagem 1 resume as tabelas criadas pelo *job* (o *script* completo está em `src/sql/01_criar_bd.sql`).

```
clubes          (codigo PK, nome, cidade, latitude, longitude, abertura, fecho, origem)
campos          (clube FK, campo, piso, coberto, iluminacao)              PK (clube, campo)
stg_reservas    (origem, clube, campo, data, inicio_min, fim_min, tipo, cliente_hash, contacto_valido)
meteo_hora      (clube, data_hora, prob_chuva, e_dia)                     PK (clube, data_hora)
ocupacao_slot   (clube, campo, data, hora, nr_reservas, conflito, coberto,
                 prob_chuva, risco_chuva, precisa_luz, sem_luz)           PK (clube, campo, data, hora)
ocupacao_diaria (clube, campo, data, slots_ocupados, slots_totais, taxa_ocupacao)
```

Listagem 1: Tabelas do esquema `hacampo`.

As horas são guardadas em **minutos desde a meia-noite** (18:30 → 1110). Desta forma, verificar sobreposições, durações e horários de abertura passa a ser aritmética simples.

4 Transformações

4.1 T0 – Previsão do tempo (API REST)

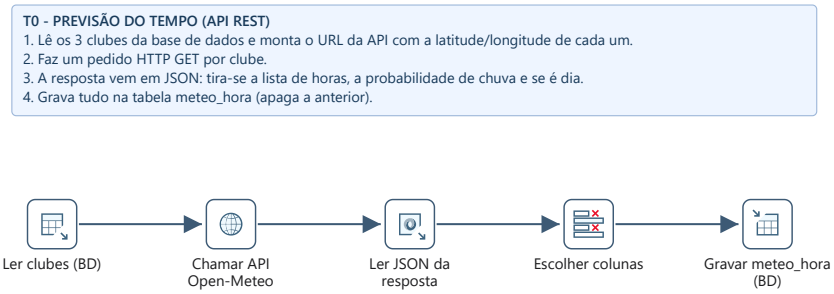


Figura 2: Transformação `t0_meteo_api.ktr`.

A transformação lê os três clubes da base de dados e monta, em SQL, o URL da API com a latitude e a longitude de cada um e com os parâmetros `${DATA_INICIO}` e `${DATA_FIM}` (substituição de variáveis ativa). O *HTTP client* faz um pedido GET por clube. A resposta é um JSON com listas paralelas (`hourly.time`, `hourly.precipitation_probability` e `hourly.is_day`), que o *JSON input* lê a partir do campo da resposta e transforma em linhas: três pedidos dão 504 linhas (3 clubes × 7 dias × 24 horas). O resultado substitui o conteúdo da tabela `meteo_hora`.

4.2 T1 – Clube A: caderno do bar (CSV)

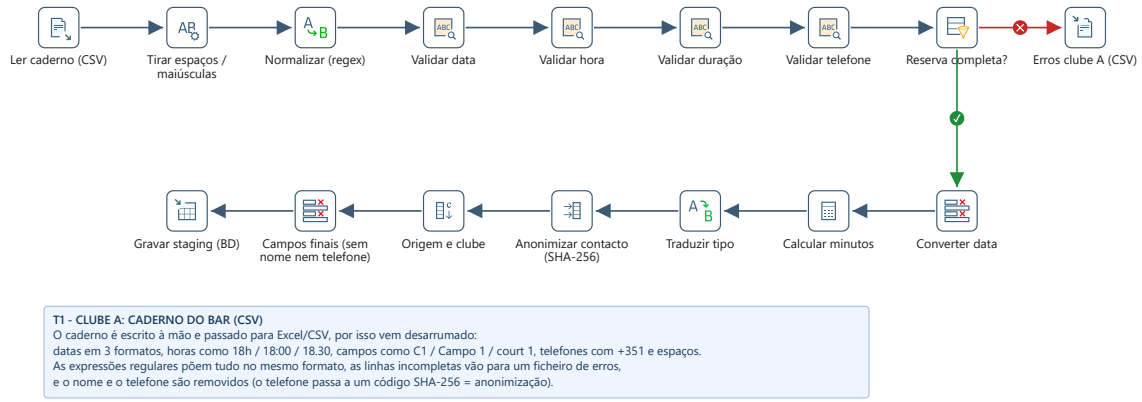


Figura 3: Transformação `t1_clube_a_csv.ktr`.

É a origem mais desorganizada. Primeiro, o *String operations* retira espaços, coloca a hora e o tipo em maiúsculas e o nome com iniciais maiúsculas. Depois, o *Replace in string* aplica as expressões regulares [8] da Tabela 5.

Tabela 5: Expressões regulares de normalização (T1).

Campo	Expressão regular	Substituição	Exemplo
data	`^(\d{2})/(\d{2})/(\d{4})$`	`$3-$2-$1`	`21/10/2026` → `2026-10-21`
data	`^(\d{2})-(\d{2})-(\d{2})$`	`20$3-$2-$1`	`21-10-26` → `2026-10-21`
hora	`^(\d{1,2})[H:.]?$`	`$1:00`	`18H` → `18:00`
hora	`^(\d{1,2})[H:.](\d{2})$`	`$1:$2`	`18h30`, `18.30` → `18:30`
campo	`\D`	(vazio)	`Campo 2`, `C2`, `court 2` → `2`
telefone	`\D` e depois `^(00351|351)(\d{9})$`	`$2`	`+351 912 345 678` → `912345678`

A seguir, quatro passos *Regex evaluation* validam os valores já normalizados e, através dos grupos de captura, extraem os números de que o cálculo precisa:

- data: `^\d{4}-\d{2}-\d{2}$`;
- hora: `^(\d{1,2}):(\d{2})$` → `h_ini`, `m_ini`;
- duração: `^(?:(\d+)\s*h)?\s*(?:(\d+)\s*(?:min|m)?)?$`, que aceita `1h`, `1h30`, `90min` e `60` → `dur_h`, `dur_m`;
- telefone português: `^(9[1236]\d{7}|2\d{8})$`.

O *Filter rows* envia as linhas incompletas para `erros_clube_a.csv`. As restantes seguem: a data passa a tipo data, o *Calculator* calcula $inicio_min = h \times 60 + m$ e $fim_min = inicio_min + duração$ (com `NVL` para as partes em falta), o *Value mapper* traduz S/P/A para singulares/pares/aula e o *Add a checksum* substitui o telefone por um código SHA-256. Este código permite contar clientes e detetar duplicados sem guardar o número (pseudonimização, no sentido do RGPD [9]). O último *Select values* deixa apenas os campos da *staging*: o nome e o telefone ficam para trás.

Resultado: 48 linhas lidas, 2 com erro (uma sem hora e outra com a data sem ano) e 46 gravadas na *staging*.

4.3 T2 – Clube B: aplicação de sócios (JSON)

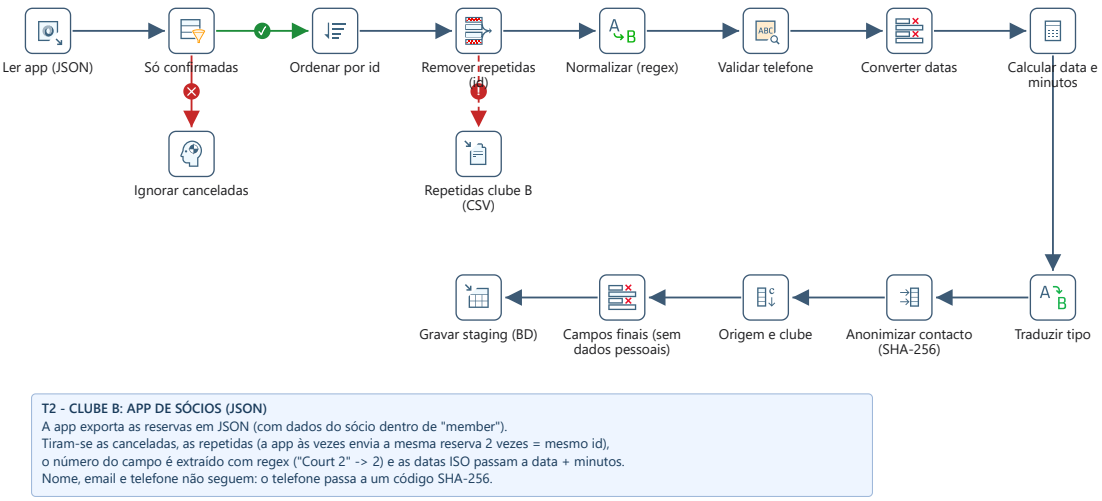


Figura 4: Transformação `t2_clube_b_json.ktr`.

O *JSON input* usa caminhos JSONPath como `$.reservas[*].member.phone` para achatar o objeto aninhado do sócio. As reservas com `status` diferente de `confirmed` são descartadas. Depois de ordenar por `id`, o *Unique rows* deteta os registos que a aplicação enviou duas vezes e redireciona-os, através de um *hop* de erro, para `repetidas_clube_b.csv`. As datas ISO (`2026-10-21T18:00:00`) são convertidas para o tipo data e o *Calculator* extrai a data sem hora (`REMOVE_TIME_FROM_DATE`), a hora (`HOUR_OF_DAY`) e o minuto (`MINUTE_OF_HOUR`). O nome, o email e o telefone não passam para a *staging*.

Resultado: 51 reservas lidas, 7 canceladas, 2 repetidas e 42 gravadas.

4.4 T3 – Clube C: agenda municipal (XML)

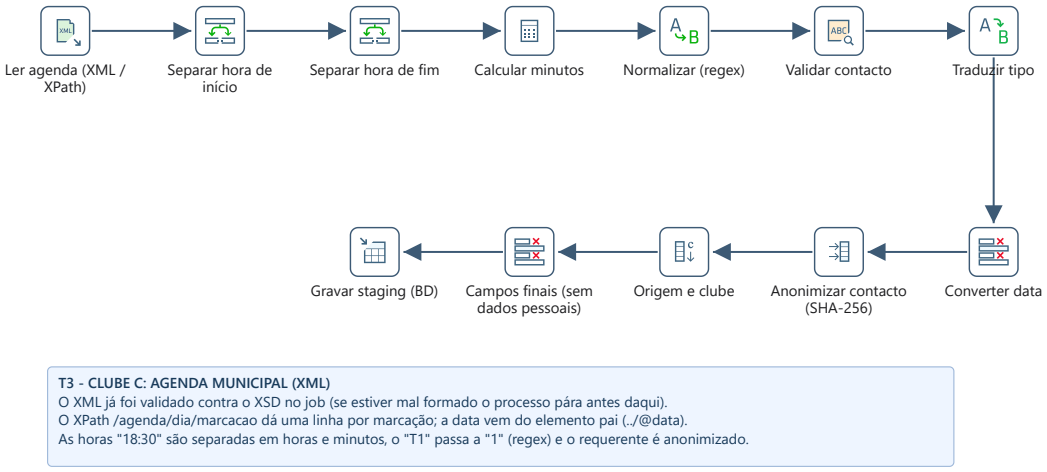


Figura 5: Transformação `t3_clube_c_xml.ktr`.

Antes desta transformação, o *job* valida o XML contra `clube_c_agenda.xsd` [7]. O XSD obriga, por exemplo, a horas no formato `HH:MM`, a campos `T1`, `T2`, … e a um tipo de marcação dentro de uma lista fechada; se o XML não cumprir estas regras, o processo pára antes de ler dados errados. O *Get data from XML* percorre `/agenda/dia/marcacao` (uma linha por marcação) e obtém a data a partir do elemento pai com o XPath `../@data`. As horas `18:30` são separadas com *Split fields*, o código `T1` passa a `1` com a expressão regular `^T` e o requerente é anonimizado.

Resultado: 32 marcações gravadas.

4.5 T4 – Consolidar e publicar

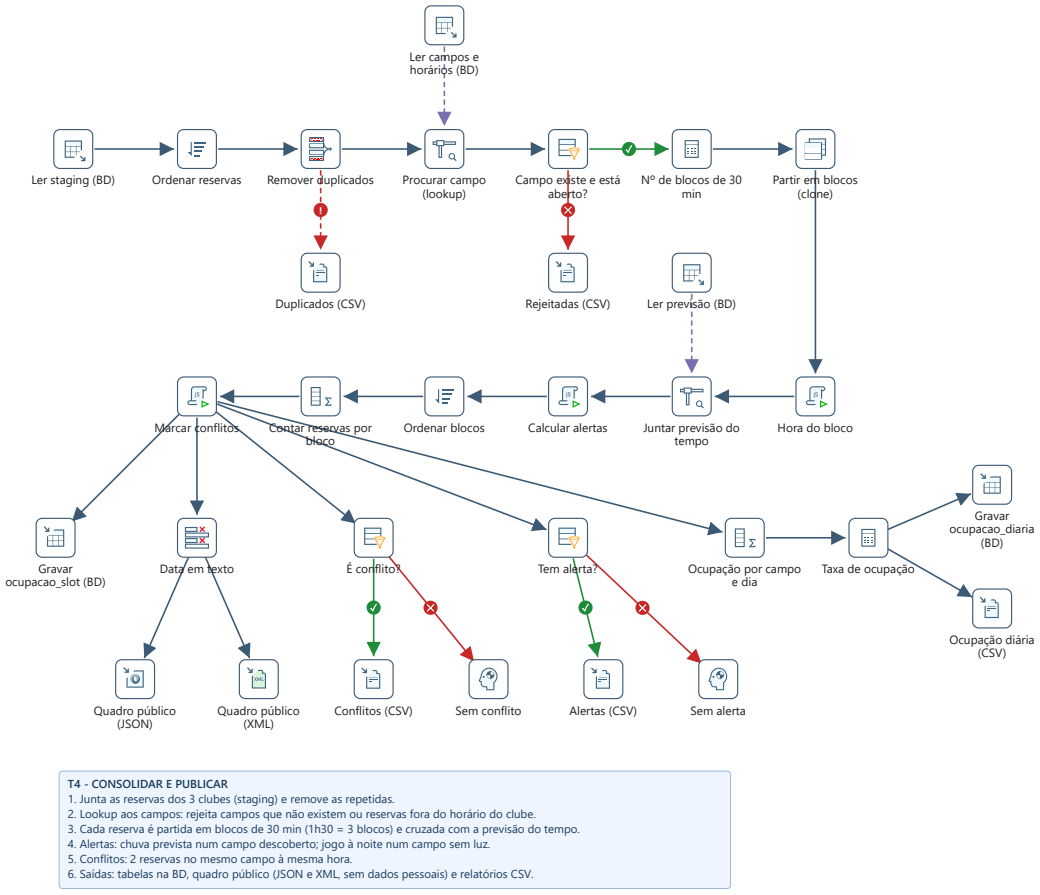


Figura 6: Transformação `t4_consolidar_publicar.ktr`.

A transformação T4 junta as reservas dos três clubes e aplica as regras de negócio:

1. **Duplicados:** lê a *staging*, ordena e remove a mesma reserva escrita duas vezes (mesmo clube, campo, data, horas e cliente), que segue para `duplicados.csv`.
2. **Validação de negócio:** um *Stream lookup* a `campos` + `clubes` acrescenta se o campo é coberto, se tem iluminação e o horário em minutos. O *Filter rows* compara campos entre si ($inicio_min \geq abertura_min$ e $fim_min \leq fecho_min$) e rejeita campos inexistentes e reservas fora do horário para `rejeitadas.csv`.
3. **Blocos de 30 minutos:** o *Calculator* calcula o número de blocos (1h30 → 3) e o *Clone row* cria uma cópia por bloco, numerada. O passo *Hora do bloco* (Listagem 2) calcula a hora de cada bloco e a chave da meteorologia.
4. **Meteorologia:** um *Stream lookup* a `meteo_hora` por clube e hora permite calcular os alertas (Listagem 3): `risco_chuva` (campo descoberto e probabilidade ≥ `LIMITE_CHUVA`), `precisa_luz` (já é noite) e `sem_luz` (é noite e o campo não tem iluminação).
5. **Conflitos:** o *Group by* conta as reservas por campo, dia e bloco; mais de uma reserva no mesmo bloco é um conflito, que segue para `conflitos.csv`.

6. **Saídas:** as linhas são copiadas em paralelo para a tabela `ocupacao_slot`, para o quadro em JSON e em XML, para os ficheiros de alertas e de conflitos e, com um segundo *Group by* e um *Calculator*, para a taxa de ocupação por campo e por dia (base de dados e CSV).

```
// cada cópia da reserva corresponde a um bloco de 30 minutos
var minuto = inicio_min + bloco * 30;
var hh = Math.floor(minuto / 60);
var mm = minuto % 60;
function dois(n) { return (n < 10 ? "0" : "") + n; }
var hora = dois(hh) + ":" + dois(mm);
// chave igual à da API: 2026-10-21T18:00
var chave_meteo = date2str(data, "yyyy-MM-dd") + "T" + dois(hh) + ":00";
```

Listagem 2: Passo *Hora do bloco* (Modified JavaScript value).

```
// os campos booleanos vêm da BD como objetos Java: compara-se com true/false
var descoberto = (coberto == false);
var tem_luz = (iluminacao == true);
// a chuva só interessa nos campos descobertos (limite em %: parâmetro LIMITE_CHUVA)
var limite = parseInt(getVariable("LIMITE_CHUVA", "50"));
var risco_chuva = (descoberto && prob_chuva != null && prob_chuva >= limite);
// e_dia = 0 -> já é noite, o campo precisa de luz
var precisa_luz = (e_dia != null && e_dia == 0);
var sem_luz = (precisa_luz && !tem_luz);
```

Listagem 3: Passo *Calcular alertas* (Modified JavaScript value).

A Tabela 6 resume uma execução completa do processo.

Tabela 6: Resultados da execução de 20 de outubro de 2026.

Indicador	**Valor**
Reservas na *staging* (3 clubes)	120
Duplicados removidos / rejeitadas (campo inexistente, fora de horas)	2 / 2
Reservas válidas → blocos de 30 minutos	116 → 316
Blocos ocupados distintos / em conflito	311 / 5
Blocos com alerta (noite num campo sem luz)	14
Blocos com chuva ≥ 50% num campo descoberto	0 (semana seca; com `LIMITE_CHUVA = 5` há 99)
Linhas de ocupação diária (campo × dia)	60

5 Jobs

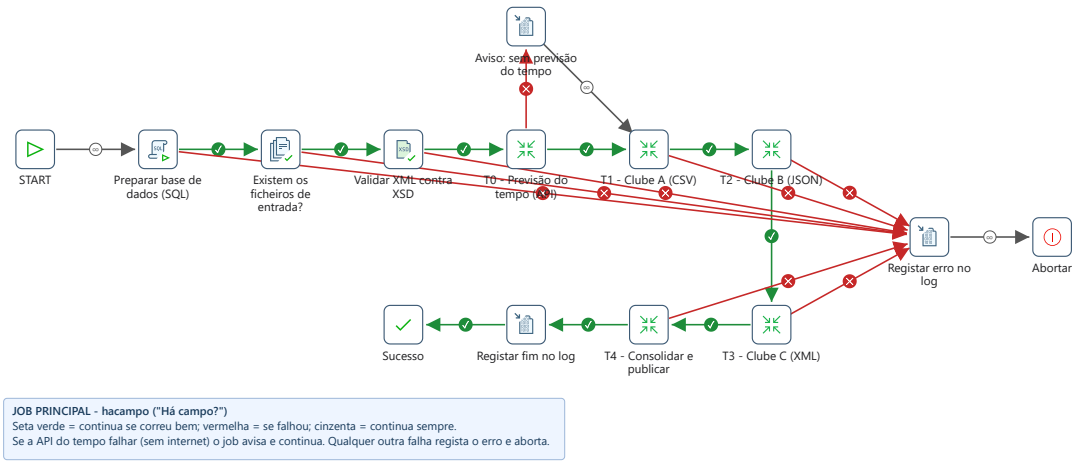


Figura 7: *Job* principal `j_hacampo.kjb`.

O *job* `j_hacampo.kjb` (Figura 7) define a ordem de execução e o que acontece quando algum passo falha:

1. **Preparar base de dados (SQL):** cria o esquema e as tabelas se não existirem, insere ou atualiza os clubes e os campos (`INSERT ... ON CONFLICT ... DO UPDATE`) e esvazia a *staging*. Pode ser executado várias vezes.
2. **Existem os ficheiros de entrada?** Verifica que os três ficheiros existem.
3. **Validar XML contra XSD.**
4. **T0 – Previsão do tempo.** Se falhar (por exemplo, sem ligação à Internet), escreve um aviso no *log* e continua com a última previsão guardada; é o único passo que pode falhar sem interromper o processo.
5. **T1, T2 e T3** carregam a *staging*; **T4** consolida e publica.
6. **Fim:** regista no *log* onde estão os resultados e termina com *Success*. Qualquer falha nos passos 1 a 3 ou em T1 a T4 segue o *hop* vermelho: o erro é registado no *log* e o *job* é abortado.

O *job* tem três parâmetros, passados às transformações: `DATA_INICIO` e `DATA_FIM` (semana pedida à API) e `LIMITE_CHUVA`. Todos os caminhos são relativos à pasta dos ficheiros (variável `${Internal.Entry.Current.Directory}`), pelo que a pasta pode ser copiada para outro computador. A Listagem 4 mostra a execução pela linha de comandos.

```
Kitchen.bat -file="...\dataintegration\j_hacampo.kjb" -level=Basic "-logfile=...\data\output\execucao.log"
```

Listagem 4: Execução do *job* com o Kitchen, com registo em ficheiro.

A Figura 8 mostra o resultado final: o quadro *"Há campo?"* a ler o ficheiro `disponibilidade.json` produzido pela T4.

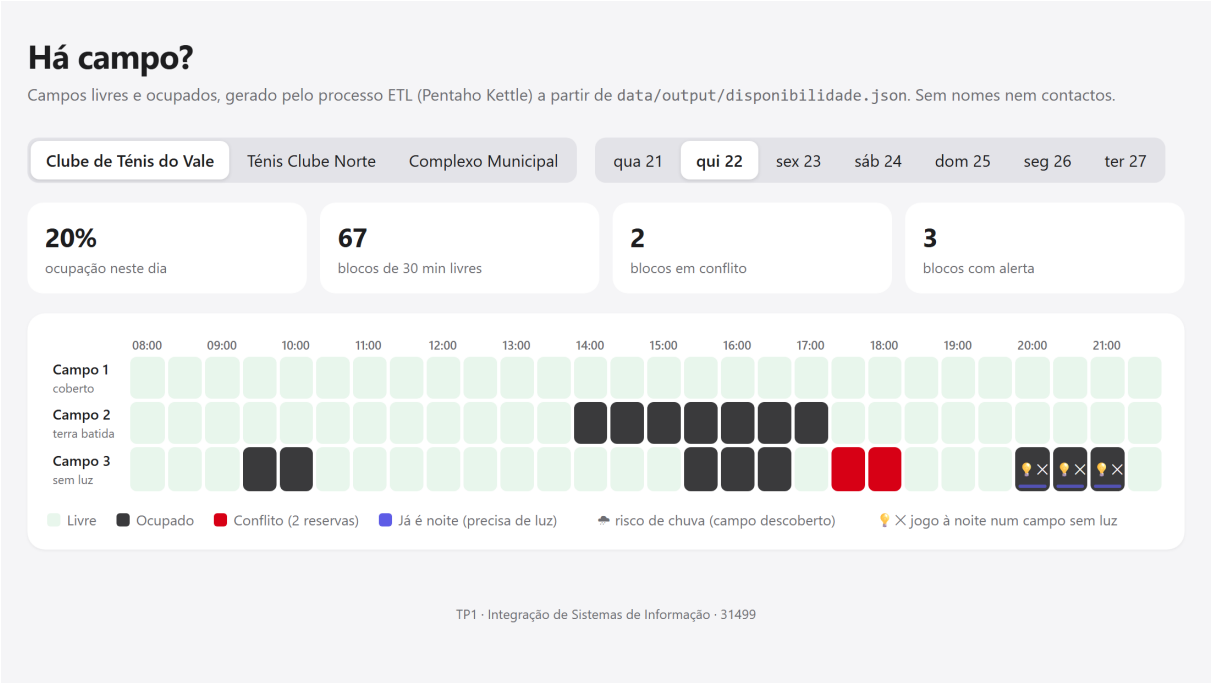


Figura 8: Quadro *"Há campo?"* (`src/quadro/index.html`): ocupado a cinzento, conflito a vermelho, jogo à noite num campo sem luz assinalado.

6 Vídeo com demonstração

O vídeo mostra a execução do *job* no Spoon, os dados de entrada com erros, os ficheiros de saída, as tabelas no PostgreSQL e o quadro final.

Vídeo de demonstração

ligação ao vídeo por inserir

7 Conclusão e trabalhos futuros

O trabalho apresenta um processo ETL completo sobre um problema real: juntar num único quadro as marcações de clubes que utilizam sistemas diferentes. O processo lê cinco origens (CSV, JSON, XML, PostgreSQL e uma API REST); limpa e normaliza os dados com expressões regulares; remove repetidos; anonimiza os dados pessoais; e aplica regras de negócio (horários, campos existentes, conflitos, luz e chuva). Os resultados ficam na base de dados e em ficheiros JSON, XML e CSV. A separação entre transformações de entrada, *staging* e consolidação facilita a integração de novos clubes.

As principais dificuldades foram as datas e horas escritas de muitas formas diferentes, a decisão sobre o que é um erro (enviado para ficheiro) e o que é apenas um aviso (a linha continua), e a anonimização sem perder a capacidade de detetar duplicados, resolvida com o código SHA-256 do contacto.

Como trabalhos futuros, propõe-se:

- executar o *job* de hora a hora (agendador do sistema ou Carte) e publicar o JSON automaticamente no site hacampo (FTP/SFTP);
- enviar por email ao clube os conflitos e alertas do dia (entrada *Mail* do *job*);
- receber as marcações em tempo real por MQTT/Node-RED, em vez de ficheiros;

- ligar o processo a clubes reais, com a respetiva autorização.

8 Referências

[1] Instituto Politécnico do Cávado e do Ave, *Integração de Sistemas de Informação – Trabalho Prático I*, enunciado, 2026/27. Material disponibilizado na unidade curricular.

[2] Instituto Politécnico do Cávado e do Ave, *Integração de Sistemas de Informação – slides das aulas* (Integração ao nível dos dados: XML e XML em C#; ETL: introdução, *Kettle Tutorial* e acesso a bases de dados), 2026/27. Material disponibilizado na unidade curricular.

[3] M. Casters, R. Bouman e J. van Dongen, *Pentaho Kettle Solutions: Building Open Source ETL Solutions with Pentaho Data Integration*. Indianapolis: Wiley, 2010.

[4] Pentaho, *Pentaho Data Integration – documentação de steps e job entries*, https://pentaho-public.atlassian.net/wiki/spaces/EAI. Consultado em 20 de outubro de 2026.

[5] Open-Meteo, *Weather Forecast API* (parâmetros `hourly`, `precipitation_probability` e `is_day`), https://open-meteo.com/en/docs. Consultado em 20 de outubro de 2026.

[6] The PostgreSQL Global Development Group, *PostgreSQL 17 Documentation* (`INSERT ... ON CONFLICT`, `CREATE TABLE IF NOT EXISTS`), https://www.postgresql.org/docs/17/. Consultado em 20 de outubro de 2026.

[7] W3C, *XML Schema Part 2: Datatypes* (`xs:pattern`, `xs:enumeration`), https://www.w3.org/TR/xmlschema-2/. Consultado em 20 de outubro de 2026.

[8] J. Goyvaerts e S. Levithan, *Regular Expressions Cookbook*. Sebastopol: O'Reilly Media, 2009.

[9] Regulamento (UE) 2016/679 do Parlamento Europeu e do Conselho (Regulamento Geral sobre a Proteção de Dados), artigos 4.º e 25.º.

Anexo A – Declaração de utilização de inteligência artificial

Foi utilizado um assistente de IA (modelo de linguagem) na análise do enunciado, no desenvolvimento do processo ETL e na redação do relatório, a partir de pedidos de escolha do tema, planeamento e execução do trabalho e organização da entrega.